iZprime Framework: Pseudocode Documentation

https://github.com/Zprime137/iZprime

January 23, 2026

Introduction

The *iZprime* framework is a high-performance C toolkit designed for prime number generation and algorithmic research. It features efficient implementations of classical and novel prime sieve algorithms, optimized for both educational use and computational tasks requiring large-scale prime enumeration.

This document provides formal pseudocode representations of the key algorithms implemented in the framework, accompanied by detailed explanations and complexity analyses. The pseudocode abstracts away low-level implementation details to emphasize the core mathematical logic and algorithmic structure of each method, making it accessible for theoretical study and practical implementation alike.

Contents

1 Basic Data Structures **2**
- 1.1 Integer Array (`INT_ARRAY`) . . . 2
- 1.2 Bitmap (`BITMAP`) . . . 2

2 Prime Sieve Algorithms **2**
- 2.1 Sieve of Eratosthenes (SoE) . . . 3
- 2.2 Segmented Sieve of Eratosthenes (SSoE) . . . 4
- 2.3 Sieve of Euler (SoEu) . . . 5
- 2.4 Sieve of Sundaram (SoS) . . . 6
- 2.5 Sieve of Atkin (SoA) . . . 7
- 2.6 Sieve-iZ (SiZ) . . . 9
- 2.7 Segmented Sieve-iZm (SiZm) . . . 11

3 Summary **14**

A Algebraic Study of the iZ Function **16**

1 Basic Data Structures

The prime sieve algorithms presented in this document rely on two fundamental data structures: an integer array (`INT_ARRAY`) for maintaining the ordered list of discovered primes, and a bitmap (`BITMAP`) for efficiently representing candidate numbers and tracking their primality status during the sieving process.

1.1 Integer Array (`INT_ARRAY`)

For storing and managing the list of prime numbers generated by sieve algorithms, we define a dynamic integer array structure called `INT_ARRAY`. This structure maintains an expandable array of integers along with metadata tracking the current count and total capacity.

A representative C definition of the `INT_ARRAY` structure is:

```
typedef struct
{
    size_t capacity;       ///< Total capacity (max count of elements) of the array
    int count;             ///< Number of elements currently stored
    int *array;            ///< Pointer to the dynamically allocated int array
} INT_ARRAY;
```

In this document, we use the following operations on `INT_ARRAY`:

- `int_array_init()`: Initializes and returns an `INT_ARRAY` object with a dynamically allocated array, with both *capacity* and *count* initialized to 0.
- `int_array_push(`*int_array*, *value*`)`: Appends the integer *value* to the end of *int_array*, expanding capacity as needed.

1.2 Bitmap (`BITMAP`)

For memory-efficient representation of candidate number ranges in sieve algorithms, we define a bitmap structure called `BITMAP`. This structure implements a compact bit array where each *bit-index* corresponds to a candidate number, and each *bit-state* (0 or 1) encodes that candidate's primality status.

A representative C definition of the `BITMAP` structure is:

```
typedef struct
{
    size_t size;        ///< Total number of bits in the bitmap
    uint8_t *data;      ///< A dynamically allocated array of bytes ((size + 7) / 8)
} BITMAP;
```

In this document, we use the following operations on `BITMAP`:

- `bitmap_init(`*size*, *initial_value*`)`: Initializes and returns a bitmap of given *size* with all bits set to *initial_value*.
- `bitmap_set_all(`*bitmap*`)`: Sets all bits in the *bitmap* to 1.
- `bitmap_get_bit(`*bitmap*, *index*`)`: Returns the bit-state at *index* in the *bitmap*.
- `bitmap_flip_bit(`*bitmap*, *index*`)`: Flips the bit-state at *index* in the *bitmap*.
- `bitmap_clear_bit(`*bitmap*, *index*`)`: Clears (sets to 0) the bit at *index* in the *bitmap*.
- `bitmap_clear_steps(`*bitmap*, *step*, *start*, *limit*`)`: Clears bits in the *bitmap* starting from *start* and incrementing by *step* until *limit* is reached.
- `bitmap_clone(`*bitmap*`)`: Creates and returns a deep copy of the given *bitmap*.

2 Prime Sieve Algorithms

This section presents detailed pseudocode for various prime sieve algorithms, accompanied by comprehensive complexity analyses. All algorithms generate an ordered list of all prime numbers up to a given limit N,

though they employ different strategies and optimizations to achieve this goal.

2.1 Sieve of Eratosthenes (SoE)

The Sieve of Eratosthenes, attributed to the ancient Greek mathematician Eratosthenes of Cyrene (c. 276–194 BCE), is one of the most elegant and fundamental algorithms for finding all prime numbers up to a specified limit N.

The algorithm operates by iteratively marking the multiples of each prime $p \leq \sqrt{N}$ as composite. Numbers that remain unmarked after all iterations are prime. This straightforward approach has been a cornerstone of computational number theory for centuries.

Algorithm 1 Sieve of Eratosthenes (SoE)

```
procedure SoE(int : n) → INT_ARRAY
                                                        ▷ 1. Initialization:
    INT_ARRAY: primes ← INT_ARRAY_INIT()                ▷ initialize primes array
    INT_ARRAY_PUSH(primes, 2)                           ▷ append 2 to primes, to focus on odd candidates
    BITMAP: sieve ← BITMAP_INIT(n + 1, 1)               ▷ create sieve bitmap with all bits set to 1
                                                        ▷ 2. Sieve Process: mark composites and collect primes
    n_sqrt ← ⌈√n⌉                                       ▷ compute root limit
    for i ← 3 to n with step 2 do                       ▷ iterate over odd candidates
        if BITMAP_GET_BIT(sieve, i) then                ▷ if i is prime
            INT_ARRAY_PUSH(primes, i)                   ▷ append i to primes
            if i < n_sqrt then                          ▷ if i is root prime
                BITMAP_CLEAR_STEPS(sieve, 2i, i^2, n)   ▷ mark its odd multiples starting from i^2
            end if
        end if
    end for
                                                        ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: Since SoE maintains a single contiguous bitmap of size $N + 1$, its auxiliary space requirement is **linear** $O(N)$. While acceptable for small to moderate values of N, this becomes prohibitive for very large limits.

Speedups: This implementation applies several well-known optimizations to improve efficiency:

- **Skip even numbers**: The algorithm treats 2 as a special case and processes only odd candidates (incrementing by 2), immediately halving the search space.
- **Start marking from** p^2: For each prime p, marking begins at p^2 rather than $2p$, since all smaller multiples have already been marked by smaller primes.
- **Mark only odd multiples**: When marking multiples of odd primes, the step size is $2p$ rather than p, skipping even composites.

After applying these optimizations, the total bit-work is estimated as:

$$O\left(\frac{N}{2}\sum_{p=3}^{\sqrt{N}}\frac{1}{p}\right).$$

Time Complexity: Since the cost per prime for marking composites is proportional to $1/p$, and the sum $\sum_{p\leq\sqrt{N}} 1/p$ grows as $O(\log\log N)$, the overall time complexity is $O(N \log\log N)$ with an improved constant factor (effectively $N/2$ instead of N for the naive implementation).

2.2 Segmented Sieve of Eratosthenes (SSoE)

The Segmented Sieve of Eratosthenes (SSoE) is a memory-efficient variant of the classical SoE algorithm. Rather than allocating a single monolithic bitmap of size N, SSoE divides the candidate range into smaller segments of size Δ (typically $\sqrt{N}$) and processes each segment sequentially.

The algorithm begins by generating all *root primes* up to $\sqrt{N}$ using the standard SoE. These root primes are then used to sieve each subsequent segment independently. This segmented approach preserves the time complexity of SoE while dramatically improving cache locality and reducing memory requirements, making it suitable for finding primes up to very large limits.

Algorithm 2 Segmented Sieve of Eratosthenes (SSoE)

```
procedure SSoE(int : n) → INT_ARRAY
                                                        ▷ 1. Initialization:
    segment_size ← ⌈√n⌉                                 ▷ compute segment size
    BITMAP: segment ← BITMAP_INIT(segment_size + 1, 1)  ▷ create segment bitmap
    INT_ARRAY: primes ← SoE(segment_size)               ▷ initialize primes array with root primes up to √n
                                                        ▷ 2. Process subsequent segments
    low ← segment_size + 1                              ▷ initialize segment bounds
    high ← low + segment_size − 1
    while low ≤ n do                                    ▷ iterate over segments
        BITMAP_SET_ALL(segment)                         ▷ reset segment bitmap
        for p ≥ 3 ∈ primes do                           ▷ sieve current segment using root primes
            if p^2 > high then break
            end if                                      ▷ no need to process larger primes
            start ← max(p^2, ⌈low/p⌉ × p)               ▷ find first multiple of p in current segment
            start ← start if start mod 2 ≠ 0 else start + p   ▷ ensure start is odd
            BITMAP_CLEAR_STEPS(segment, 2p, start − low, high − low)   ▷ mark multiples
        end for
        for i ← low to high with step 2 do              ▷ collect primes from segment
            if BITMAP_GET_BIT(segment, i − low) then    ▷ if unmarked, it's prime
                INT_ARRAY_PUSH(primes, i)
            end if
        end for
        low ← low + segment_size                        ▷ move to next segment
        high ← min(high + segment_size, n)
    end while
                                                        ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: SSoE requires only $O(\sqrt{N})$ auxiliary space for the segment bitmap, plus $O(\pi(\sqrt{N}))$ space for storing the root primes (where $\pi(x)$ denotes the prime counting function). This represents a significant improvement over SoE's $O(N)$ space requirement, enabling the algorithm to handle limits far beyond what would fit in main memory using a monolithic sieve.

Speedups: SSoE inherits all the optimizations from SoE: skipping even numbers, starting marking from p^2, and using step sizes of $2p$. The key additional advantage lies in improved cache locality—by processing the sieve in small, cache-resident segments, the algorithm achieves dramatically faster memory access patterns compared to the monolithic SoE bitmap. For each segment, we only use primes up to $\sqrt{high}$ for marking, where $high$ is the upper bound of the current segment.

Time Complexity: The time complexity remains $O(N \log \log N)$, matching SoE asymptotically. However, in practice, SSoE often runs faster due to superior cache utilization, especially for large N where the monolithic bitmap would thrash the cache.

2.3 Sieve of Euler (SoEu)

The Sieve of Euler, also known as the Linear Sieve, is an elegant prime-finding algorithm that ensures each composite number is marked exactly once. Unlike the Sieve of Eratosthenes, which may mark the same composite multiple times (for example, 15 is marked when sieving with both 3 and 5), the Sieve of Euler guarantees that each composite n is marked only by its smallest prime factor (SPF).

This single-marking property makes the algorithm theoretically linear in the number of operations performed. However, in practice, it may be slower than optimized versions of SoE due to less predictable memory access patterns and the overhead of the inner loop over all collected primes.

Algorithm 3 Sieve of Euler (SoEu)

```
1:  procedure SoEu(int : n) → INT_ARRAY
2:                                                  ▷ 1. Initialization:
3:      INT_ARRAY: primes ← INT_ARRAY_INIT()        ▷ initialize primes array
4:      INT_ARRAY_PUSH(primes, 2)                   ▷ add 2 to primes
5:      BITMAP: sieve ← BITMAP_INIT(n + 1, 1)       ▷ create sieve bitmap
6:                                  ▷ 2. Sieve Process: collect primes and mark composites
7:      for i ← 3 to n with step 2 do               ▷ iterate over odd candidates
8:          if BITMAP_GET_BIT(sieve, i) then        ▷ if i is prime
9:              INT_ARRAY_PUSH(primes, i)           ▷ add i to primes
10:         end if
11:         for p > 2 ∈ primes do                   ▷ mark multiples using collected primes
12:             if p × i > n then break             ▷ stop if product exceeds n
13:             end if
14:             BITMAP_CLEAR_BIT(sieve, p × i)      ▷ mark p × i as composite
15:             if i mod p = 0 then break           ▷ crucial: stop at smallest prime factor
16:             end if
17:         end for
18:     end for
19:                                                 ▷ 3. Return primes
20:     return primes
21: end procedure
```

Complexity Analysis

Space Complexity: The algorithm requires $O(N)$ space for the sieve bitmap and $O(\pi(N))$ space for storing the progressively collected primes list.

Speedups: The key optimization is the inner loop termination condition at line 13: when $i \bmod p = 0$, we know that p is the smallest prime factor of i. Therefore, any composite of the form $i \times q$ for primes $q > p$ will have a smaller prime factor (namely p), and will be marked later when that smaller factor is encountered. This crucial insight ensures each composite is marked exactly once.

The algorithm skips even numbers by starting at 3 and incrementing by 2, processing only odd candidates. The total bit-work for marking all odd composites exactly once is:

$$O\left(\frac{N}{2}\right).$$

Time Complexity: The algorithm achieves true $O(N)$ time complexity since each candidate number is visited at most once during the marking phase. However, the constant factors are typically higher than optimized SoE implementations due to the nested loop structure over all collected primes and less cache-friendly memory access patterns.

2.4 Sieve of Sundaram (SoS)

The Sieve of Sundaram is an elegant algorithm for finding all prime numbers up to a specified limit N. Discovered by Indian mathematician S. P. Sundaram in 1934, it operates on a transformed space of indices that naturally correspond to odd numbers.

The algorithm works by eliminating numbers of the form $i + j + 2ij$ from consideration, where $1 \leq i \leq j$ and $i + j + 2ij \leq k = \lfloor (N-1)/2 \rfloor$. After this elimination process, the remaining indices i in the range $[1, k]$ correspond to primes via the transformation $2i + 1$.

This approach is conceptually elegant and naturally produces only odd primes (with 2 handled as a special case). The algorithm operates on a reduced space of size approximately $N/2$, similar to the optimization in SoE that skips even numbers.

Algorithm 4 Sieve of Sundaram (SoS)

```
procedure SoS(int : n) → INT_ARRAY
                                                    ▷ 1. Initialization:
    INT_ARRAY: primes ← INT_ARRAY_INIT()            ▷ initialize primes array
    INT_ARRAY_PUSH(primes, 2)                       ▷ add 2 (special case)
    k ← ⌊(n−1)/2⌋                                   ▷ compute the size of the odd set up to n
    BITMAP: sieve ← BITMAP_INIT(k, 1)               ▷ create sieve bitmap
                                                    ▷ 2. Sieve Process: mark non-primes and collect primes
    n_sqrt ← ⌈√n⌉                                   ▷ root limit
    for i ← 1 to k do                               ▷ iterate through Sundaram space
        if BITMAP_GET_BIT(sieve, i) then            ▷ if i is unmarked
            p ← 2i + 1                              ▷ compute prime from index
            INT_ARRAY_PUSH(primes, p)               ▷ add prime
            if p < n_sqrt then                      ▷ if p is root prime
                x_p ← p × i + i                     ▷ first composite index: corresponds to p^2
                BITMAP_CLEAR_STEPS(sieve, p, x_p, k)  ▷ mark composites additively with step p
            end if
        end if
    end for
                                                    ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: The algorithm requires **linear** $O(N/2)$ space for the sieve bitmap, which is approximately half the size of the classical SoE bitmap.

Speedups: The classical Sieve of Sundaram uses a multiplicative approach to mark composites, targeting numbers of the form $i + j + 2ij$, which yields $O(N \log N)$ time complexity. However, this implementation leverages the Xp-Identity (described in Section 2.6) to transform the marking process into an additive operation with step size p.

This optimization, which recognizes that odd composites in the Sundaram space can be marked additively,

reduces the bit-work to:

$$O\left(\frac{N}{2}\sum_{p=3}^{\sqrt{N}}\frac{1}{p}\right),$$

matching the performance of optimized SoE.

Time Complexity: With the additive marking optimization, the time complexity becomes $O(N \log\log N)$ with constant factors similar to the basic SoE implementation.

2.5 Sieve of Atkin (SoA)

The Sieve of Atkin is a modern, theoretically optimal algorithm for finding all prime numbers up to a specified limit N. Developed by A. O. L. Atkin and Daniel J. Bernstein in 2004, it employs a fundamentally different approach from the classical Eratosthenes sieve, based on deep results from algebraic number theory involving quadratic forms.

The algorithm proceeds in two main phases:

1. **Preprocessing**: Mark candidates by testing specific modular arithmetic conditions based on three quadratic forms ($4x^2+y^2$, $3x^2+y^2$, and $3x^2-y^2$). Numbers satisfying these conditions an odd number of times are potentially prime.
2. **Sieving**: Eliminate composites by clearing all odd multiples of squares of primes.

The bit-flipping mechanism in the first phase accounts for numbers that satisfy multiple quadratic forms—a number is prime if and only if it satisfies the conditions an odd number of times.

Algorithm 5 Sieve of Atkin (SoA)

```
procedure SoA(int : n) → INT_ARRAY
                                                                    ▷ 1. Initialization:
    INT_ARRAY: primes ← INT_ARRAY_INIT()                            ▷ initialize primes array
    INT_ARRAY_PUSH(primes, 2)                                       ▷ add 2 and 3 as special cases
    INT_ARRAY_PUSH(primes, 3)
    BITMAP: sieve ← BITMAP_INIT(n + 1, 0)                           ▷ create sieve bitmap, initially all 0
                                ▷ 2. Preprocessing: Mark potential primes using Atkin conditions
    for x ← 1 while 4x^2 < n do                                     ▷ Condition 1: 4x^2 + y^2 ≡ 1 (mod 12) or 5 (mod 12)
        for y ← 1 while 4x^2 + y^2 ≤ n do
            b ← 4x^2 + y^2
            if b mod 12 = 1 or b mod 12 = 5 then
                BITMAP_FLIP_BIT(sieve, b)                           ▷ flip bit
            end if
        end for
    end for
    for x ← 1 while 3x^2 < n do                                     ▷ Condition 2: 3x^2 + y^2 ≡ 7 (mod 12)
        for y ← 1 while 3x^2 + y^2 ≤ n do
            b ← 3x^2 + y^2
            if b mod 12 = 7 then
                BITMAP_FLIP_BIT(sieve, b)
            end if
        end for
    end for
    for x ← 1 while 2x^2 < n do                                     ▷ Condition 3: 3x^2 − y^2 ≡ 11 (mod 12) and x > y
        for y ← x − 1 down to 1 do
            b ← 3x^2 − y^2
            if b > n then break
            end if
            if b mod 12 = 11 then
                BITMAP_FLIP_BIT(sieve, b)
            end if
        end for
    end for
                                ▷ 3. Sieve: Eliminate composites by removing multiples of squares
    n_sqrt ← ⌈√n⌉
    for p ← 5 to n_sqrt with step 2 do                              ▷ iterate over odd numbers ≥ 5
        if BITMAP_GET_BIT(sieve, p) then                            ▷ if p survived, it's prime
            BITMAP_CLEAR_STEPS(sieve, 2p^2, p^2, n)                 ▷ clear odd multiples of p^2
        end if
    end for
                                                                    ▷ 4. Collect remaining primes
    for p ← 5 to n with step 2 do
        if BITMAP_GET_BIT(sieve, p) then
            INT_ARRAY_PUSH(primes, p)
        end if
    end for
                                                                    ▷ 5. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: The algorithm requires $O(N)$ space for the sieve bitmap.

Speedups: The Sieve of Atkin's primary advantage stems from its use of quadratic forms to identify prime candidates more directly than the Eratosthenes approach. By testing specific modular conditions based on well-chosen quadratic forms, many numbers are classified as prime or composite without the repeated marking inherent to the Eratosthenes sieve.

The bit-flipping mechanism elegantly handles numbers that satisfy multiple quadratic forms. The final elimination phase is remarkably efficient, removing only odd multiples of squares of primes (rather than all multiples), resulting in a cost per prime of approximately $N/(2p^2)$.

The total bit-work can be approximated as:

$$O\left(N\left(\frac{1}{4}+\frac{1}{6}+\frac{1}{12}\right)+\frac{N}{2}\sum_{p=5}^{\sqrt{N}}\frac{1}{p^2}\right)=O(N),$$

where the first term represents the quadratic form tests and the second term (which converges) represents the square-multiple elimination.

Time Complexity: The theoretical time complexity is **linear** $O(N)$, which is asymptotically superior to SoE's $O(N \log\log N)$. However, in practice for typical ranges (up to 10^{12}), highly optimized implementations of SoE or its segmented variants often outperform SoA due to simpler operations, better cache utilization, and lower constant factors. The Sieve of Atkin's advantage becomes more pronounced for extremely large limits where the asymptotic improvement dominates.

2.6 Sieve-iZ (SiZ)

The Sieve-iZ (SiZ) algorithm employs a wheel factorization optimization based on modulus 6. By recognizing that all primes greater than 3 must be of the form $6k \pm 1$, the algorithm restricts its search to the candidate space $i\mathbb{Z} = \{z \in \mathbb{N} : z \equiv \pm 1 \pmod 6\}$, automatically excluding all multiples of 2 and 3.

This approach reduces space requirements from N to approximately $N/3$ and lowers the cost per prime for marking composites to $N/(3p)$. The algorithm maintains two separate bitmaps—one for each residue class modulo 6—and employs the Xp-Identity for efficient, additive composite marking.

We begin with the formal definitions and notations that underpin the SiZ family of algorithms:

Definition 2.1 (The $i\mathbb{Z}$ set)**.** The set of all natural numbers coprime to 6, expressed as the union of two disjoint residue classes modulo 6:

$$i\mathbb{Z} \;=\; \{z \in \mathbb{N} : z \equiv \pm 1 \pmod 6\} \;=\; i\mathbb{Z}^- \dot\cup\, i\mathbb{Z}^+,$$

where

$$i\mathbb{Z}^- = \{6x-1 : x \in \mathbb{N}\} \quad \text{and} \quad i\mathbb{Z}^+ = \{6x+1 : x \in \mathbb{N}\},$$

with cardinality $|i\mathbb{Z}| = \frac{1}{3}|\mathbb{N}|$. Note that all primes $p > 3$ belong to $i\mathbb{Z}$.

To map between indices x and values of the form $6x \pm 1$, we define the iZ function:

Definition 2.2 (The iZ function)**.**

$$\boxed{iZ(x \in \mathbb{N},\ i \in \{\pm 1\}) = 6x + i}.$$

Here, x is the *index* within a residue class, and i specifies which residue class ($i = -1$ for $i\mathbb{Z}^-$ and $i = +1$ for $i\mathbb{Z}^+$).

X-Arrays. When sieving $i\mathbb{Z}$ up to a limit N, we maintain two separate bitmaps, one per residue class, each of length $x_n = \lceil N/6 \rceil$ bits:

- `X5[`x`]` encodes candidates in the $i\mathbb{Z}^-$ lane (numbers of the form $6x - 1$),
- `X7[`x`]` encodes candidates in the $i\mathbb{Z}^+$ lane (numbers of the form $6x + 1$).

The Xp-Identity: Composite Marking within $i\mathbb{Z}$. The Xp-Identity provides a modular, index-based formula for marking all composites of a given prime p within the $i\mathbb{Z}$ space using simple additive steps. Derived from Lemma A.1 (Appendix A).

Definition 2.3 (Xp-Identity)**.** Let $p = iZ(x_p, i_p) \in \mathbb{P}$ be a prime. The indices of composites of p in the X-Arrays are given by:

$$\begin{aligned} &\texttt{X5:} \quad \left\{\, p\,x_q - i_p\,x_p \;:\; x_q \geq x_p \in \mathbb{N} \,\right\} \quad (\text{composites of } p \text{ in } i\mathbb{Z}^-), \\ &\texttt{X7:} \quad \left\{\, p\,x_q + i_p\,x_p \;:\; x_q \geq x_p \in \mathbb{N} \,\right\} \quad (\text{composites of } p \text{ in } i\mathbb{Z}^+). \end{aligned}$$

The constraint $x_q \geq x_p$ ensures we start marking from $p \geq q$, avoiding duplicate marking of composites whose smaller cofactor q would have already been processed.

Algorithm 6 Sieve-iZ (SiZ)

```
procedure SiZ(int : n) → INT_ARRAY
                                                        ▷ 1. Initialization
    INT_ARRAY: primes ← INT_ARRAY_INIT()                ▷ initialize primes array and add 2 and 3
    INT_ARRAY_PUSH(primes, 2)
    INT_ARRAY_PUSH(primes, 3)
    x_n ← ⌊(n+1)/6⌋ + 1                                 ▷ iZ upper bound
    BITMAP: x5 ← BITMAP_INIT(x_n, 1)                    ▷ create bitmap x5 for representing iZ− candidates
    BITMAP: x7 ← BITMAP_INIT(x_n, 1)                    ▷ create bitmap x7 for representing iZ+ candidates
                                                        ▷ 2. Sieve: mark composites and collect primes
    n_sqrt ← ⌈√n⌉                                       ▷ root limit
    for x ← 1 to x_n do                                 ▷ iterate over iZ candidates
        if BITMAP_GET_BIT(x5, x) then                   ▷ if iZ− prime
            p ← iZ(x, −1)                               ▷ compute p
            INT_ARRAY_PUSH(primes, p)                   ▷ append p to primes
            if p < n_sqrt then                          ▷ mark multiples of p in both x5 and x7 using the Xp identity
                BITMAP_CLEAR_STEPS(x5, p, p · x + x, x_n)
                BITMAP_CLEAR_STEPS(x7, p, p · x − x, x_n)
            end if
        end if
        if BITMAP_GET_BIT(x7, x) then                   ▷ if iZ+ prime, similar process with i_p = 1
            p ← iZ(x, 1)
            INT_ARRAY_PUSH(primes, p)
            if p < n_sqrt then
                BITMAP_CLEAR_STEPS(x5, p, p · x − x, x_n)
                BITMAP_CLEAR_STEPS(x7, p, p · x + x, x_n)
            end if
        end if
    end for
                                                        ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: SiZ maintains two bitmaps, each of size $\lceil N/6 \rceil$ bits, for a total auxiliary space of **linear** $O(N/3)$. This represents a $3\times$ improvement over the classical SoE.

Time Complexity: Using additive marking within the $i\mathbb{Z}$ space via the Xp-Identity, where candidates are of the form $6x \pm 1$, the total bit-work is:

$$O\left(\frac{N}{3}\sum_{p=5}^{\sqrt{N}}\frac{1}{p}\right),$$

yielding a time complexity of $O(N \log\log N)$ with significantly improved constant factors compared to classical SoE (effectively $N/3$ instead of $N/2$ for the optimized SoE).

2.7 Segmented Sieve-iZm (SiZm)

The Segmented Sieve-iZm (SiZm) algorithm represents the culmination of optimizations in this framework, extending the $i\mathbb{Z}$ basic-wheel structure ($6x \pm 1$) with another layer of extended wheel we call iZ-Matrix (iZm), resulting in a highly cache-efficient algorithm with minimal memory footprint and excellent constant factors.

Key Ideas, Definitions, and Notations

Let $vx = VX_k$ be the product of the first k primes greater than 3. The iZm structure is a two-dimensional representation of the $i\mathbb{Z}$ space with fixed horizontal width vx and unbounded vertical height vy. Each cell in the iZm corresponds to a candidate number in $i\mathbb{Z}$, mapped as follows:

- **iZm5** (representing $i\mathbb{Z}^-$): a candidate z at $[x, y]$ maps to $iZ(vx \cdot y + x,\ -1)$.
- **iZm7** (representing $i\mathbb{Z}^+$): a candidate z at $[x, y]$ maps to $iZ(vx \cdot y + x,\ +1)$.

In the SiZm algorithm, we construct the iZm with maximum horizontal width VX_6 (the product of the first 6 primes greater than 3: 5, 7, 11, 13, 17, and 19). This choice balances cache efficiency and can be tuned based on the hardware architecture. This segmentation strategy provides several key advantages:

- **Cache efficiency**: Irrespective of N, segments fit comfortably in CPU cache, dramatically improving memory access patterns, and reducing latency compared to monolithic bitmaps.
- **Wheel factorization**: Pre-sieving small primes (5, 7, 11, 13, 17, 19) that divide VX_6 in a base template eliminates the most expensive marking operations.
- **Scalability**: The algorithm can handle arbitrarily large limits N without requiring massive contiguous memory allocation.

To maintain clarity and modularity, we define several helper procedures that encapsulate reusable components of the algorithm:

Compute optimal segment size *VX*. The `iZm_compute_limited_vx` procedure calculates the maximum segment size VX that does not exceed $\lfloor (n+1)/6 \rfloor$, computed as the product of the first k primes greater than 3. This ensures the segment size is appropriate for the given limit.

```
INT_ARRAY: s_primes = [5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]   ▷ global list of primes > 3
procedure IZM_COMPUTE_LIMITED_VX(n, k)
    x_n ← ⌊(n+1)/6⌋
    vx ← 1
    i ← 0
    for p ∈ s_primes while i < k do                ▷ iterate over first k primes greater than 3
        if vx × p > x_n then break                 ▷ stop if product exceeds x_n
        end if
        vx ← vx × p
        i ← i + 1
```

```
    end for
    return vx
end procedure
```

Construct pre-sieved base bitmaps. The `iZm_construct_vx_base` procedure sieves the base bitmaps for a segment of size VX using all primes that divide VX. These pre-sieved base bitmaps serve as templates for initializing each segment's active bitmaps, eliminating the need to mark these small primes during individual segment processing.

```
procedure IZM_CONSTRUCT_VX_BASE(vx, base_x5, base_x7)
    for p ∈ s_primes such that p | vx do                    ▷ iterate over primes dividing vx
        ip ← 1 if p mod 6 == 1 else −1
        x_p ← ⌊(p+1)/6⌋
        bitmap ← base_x5 if ip == −1 else base_x7           ▷ select bitmap containing p
        BITMAP_CLEAR_BIT(bitmap, x_p)                       ▷ mark p itself as composite within the template
                                                            ▷ Mark composites of p in base bitmaps using the Xp-Identity
        BITMAP_CLEAR_STEPS(base_x5, p, p · x_p − ip · x_p, vx)
        BITMAP_CLEAR_STEPS(base_x7, p, p · x_p + ip · x_p, vx)
    end for
end procedure
```

The *Xp* solver for segmented marking. The `iZm_solve_for_xp` procedure computes the first composite index of a prime p within a given segment y, based on the Xp-Identity. This solver is crucial for determining where to begin marking composites when processing each segment.

```
procedure IZM_SOLVE_FOR_XP(m_id, p, vx, y)
    ip ← 1 if p mod 6 == 1 else −1
    xp ← ⌊(p+1)/6⌋
    if y == 0 then
        return p · xp + m_id · ip · xp                      ▷ first composite index in segment 0
    end if
    xp ← xp if m_id == ip else p − xp                       ▷ normalize xp for cross-lane marking
    x ← p − ((vx · y − xp) mod p)                           ▷ solve for first hit in segment y
    return x
end procedure
```

Algorithm 7 Segmented Sieve-iZm (SiZm)

```
procedure SiZm(int : n) → INT_ARRAY
                                                                    ▷ 1. Initialization
    vx ← iZm_compute_limited_vx(n, 6)                               ▷ compute segment size, up to VX_6
                ▷ Initialize primes with SiZ(iZ(vx, 1)) to sieve first segment and collect root primes
    INT_ARRAY: primes ← SiZ(iZ(vx, 1))
                                        ▷ Initialize base bitmaps, pre-sieved by primes dividing vx
    BITMAP: base_x5 ← bitmap_init(vx, 1)                            ▷ create base bitmap for iℤ⁻
    BITMAP: base_x7 ← bitmap_init(vx, 1)                            ▷ create base bitmap for iℤ⁺
    iZm_construct_vx_base(vx, base_x5, base_x7)                     ▷ pre-sieve base bitmaps
                                        ▷ 2. Process remaining segments for y ∈ [1, y_limit]
    x_n ← ⌊(n+1)/6⌋ + 1                                             ▷ iℤ upper bound
    y_limit ← ⌊x_n/vx⌋                                              ▷ number of full segments
    for y ← 1 to y_limit (inclusive) do
                                        ▷ a. Initialize active bitmaps from pre-sieved base templates
        BITMAP: x5 ← bitmap_clone(base_x5)                          ▷ clone for active iℤ⁻ sieve
        BITMAP: x7 ← bitmap_clone(base_x7)                          ▷ clone for active iℤ⁺ sieve
                                        ▷ b. Mark composites using root primes in current segment
        x_limit ← vx if (y < y_limit) else x_n mod vx               ▷ handle partial last segment
        yvx ← y × vx                                                ▷ base index offset for current segment
        root_limit ← √(6 · (yvx + x_limit)) + 1                     ▷ local root limit
        for p ≥ 5 ∈ primes do                                       ▷ iterate over primes > 3
            if p | vx then continue                                 ▷ skip pre-sieved primes
            end if
            if p > root_limit then break                            ▷ no need for larger primes
            end if
                                        ▷ Mark composites of p in current segment using Xp solver
            bitmap_clear_steps(x5, p, iZm_solve_for_xp(−1, p, vx, y), x_limit)
            bitmap_clear_steps(x7, p, iZm_solve_for_xp(1, p, vx, y), x_limit)
        end for
                                        ▷ c. Collect primes from current segment
        for x ← 1 to x_limit (inclusive) do
            if bitmap_get_bit(x5, x) then                           ▷ unmarked index in iℤ⁻
                int_array_push(primes, iZ(yvx + x, −1))
            end if
            if bitmap_get_bit(x7, x) then                           ▷ unmarked index in iℤ⁺
                int_array_push(primes, iZ(yvx + x, 1))
            end if
        end for
    end for
                                                                    ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: Rather than using an arbitrary segment size of $\sqrt{N}$ as in traditional segmented sieves, SiZm decouples the segment size from N and employs a fixed maximum wheel size:

$$VX_6 = 5 \times 7 \times 11 \times 13 \times 17 \times 19 = 1{,}616{,}615.$$

Beyond the $O(\pi(\sqrt{N}))$ space for storing root primes, each segment requires two bitmaps of maximum size VX_6 (approximately 1.6 million bits $\approx$ 200 KB each), yielding a space complexity of $O(\pi(\sqrt{N})) + O(1)$.

Crucially, this space requirement is *independent of* N, ensuring that segments remain cache-resident even for extremely large limits.

Speedups: Building upon the SiZ optimizations, SiZm achieves further improvements by pre-sieving the small primes that divide VX_6 (namely 5, 7, 11, 13, 17, and 19) in the wheel structure. Since these small primes contribute disproportionately to the total marking cost (due to their high frequency), eliminating them from the per-segment marking phase yields substantial performance gains.

After pre-sieving these primes, the remaining bit-work is estimated as:

$$O\left(\frac{N}{3}\sum_{p=23}^{\sqrt{N}}\frac{1}{p}\right)+O(1),$$

where the $O(1)$ term represents the one-time cost of wheel construction, which is negligible compared to the total work.

Time Complexity: The time complexity remains $O(N \log\log N)$, but with dramatically improved constant factors. In practice, SiZm often outperforms all other algorithms in this document for large limits, due to the combination of reduced search space, pre-sieving of expensive small primes, and excellent cache locality.

3 Summary

Table 1 provides a comprehensive comparison of the prime sieve algorithms presented in this document. The algorithms exhibit a range of trade-offs between space efficiency, time complexity, and practical performance.

Key observations:

- **Space vs. Time**: While SoEu and SoA achieve linear $O(N)$ time complexity, they require $O(N)$ space. In contrast, SSoE and SiZm achieve similar practical performance with dramatically reduced space requirements.
- **Segmentation benefits**: SSoE and SiZm demonstrate that segmentation provides substantial practical advantages through improved cache locality, even when asymptotic complexity remains unchanged.
- **Wheel factorization**: The progression from SoE ($O(N)$ space) to SiZ ($O(N/3)$ space) to SiZm ($O(1)$ space beyond root primes) shows the power of wheel factorization combined with segmentation.
- **Practical considerations**: Despite superior asymptotic complexity, SoEu and SoA often underperform optimized implementations of SoE-family algorithms in practice due to cache effects and instruction-level overhead.

Algorithm	**Space Complexity**	**Time Complexity**	**Bit-Work**
SoE	$O(N)$	$O(N \log\log N)$	$O\left(\frac{N}{2}\sum_{p=3}^{\sqrt{N}}\frac{1}{p}\right)$
SSoE	$O(\pi(\sqrt{N})+\sqrt{N})$	$O(N \log\log N)$	$O\left(\frac{N}{2}\sum_{p=3}^{\sqrt{N}}\frac{1}{p}\right)$
SoEu	$O(N)$	$O(N)$	$O\left(\frac{N}{2}\right)$
SoS	$O(N/2)$	$O(N \log\log N)$	$O\left(\frac{N}{2}\sum_{p=3}^{\sqrt{N}}\frac{1}{p}\right)$
SoA	$O(N)$	$O(N)$	$O\left(\frac{N}{2}+\frac{N}{2}\sum_{p=5}^{\sqrt{N}}\frac{1}{p^2}\right)$
SiZ	$O(N/3)$	$O(N \log\log N)$	$O\left(\frac{N}{3}\sum_{p=5}^{\sqrt{N}}\frac{1}{p}\right)$
SiZm	$O(\pi(\sqrt{N}))+O(1)$	$O(N \log\log N)$	$O\left(\frac{N}{3}\sum_{p=23}^{\sqrt{N}}\frac{1}{p}\right)+O(1)$

Table 1: Comprehensive comparison of prime sieve algorithms. Space complexity represents auxiliary space beyond the output. Bit-work estimates reflect the number of bit operations performed during sieving.

A Algebraic Study of the iZ Function

This appendix provides the mathematical foundation for the Xp-Identity used in the SiZ and SiZm algorithms. We establish a general lemma about products in the $i\mathbb{Z}$ space and derive the key equivalence relation that enables efficient composite marking.

Lemma A.1. *Let $m = 6$ be the modulus defining the $i\mathbb{Z}$ space. Let $\mathcal{I} = \{\pm 1\}$ be the set of residues coprime to m, which is closed under multiplication modulo m. Let $iZ(x \in \mathbb{N}, i \in \mathcal{I}) = mx + i$.*

For any $p = iZ(x_p, i_p)$ and $q = iZ(x_q, i_q)$ in $i\mathbb{Z}$, their product $z = pq$ can be expressed as $z = iZ(x_z, i_z)$, where

$$x_z = mx_p x_q + i_p x_q + i_q x_p \quad \text{and} \quad i_z = i_p i_q \in \mathcal{I}.$$

Consequently, for each prime factor p that divides z, the following equivalence holds:

$$p \mid z \quad \Longleftrightarrow \quad x_z \equiv \left(\frac{i_z}{i_p}\right) x_p \pmod{p}.$$

This equivalence forms the basis of the Xp-Identity, enabling efficient additive marking of composites in the $i\mathbb{Z}$ space.

Proof. We expand the product $z = pq$ by substituting the iZ representations:

$$z = pq = (mx_p + i_p)(mx_q + i_q) = m^2 x_p x_q + m i_p x_q + m i_q x_p + i_p i_q.$$

Factoring out m from the first three terms:

$$z = m(mx_p x_q + i_p x_q + i_q x_p) + i_p i_q = mx_z + i_z,$$

which confirms that $z \in i\mathbb{Z}$ with $z = iZ(x_z, i_z)$, where:

- The *product residue* $i_z \in \mathcal{I}$ is determined by:

$$i_z = i_p i_q = \begin{cases} +1 & \text{if } i_p = i_q, \\ -1 & \text{if } i_p \neq i_q. \end{cases}$$

- The *product index* x_z can be rewritten in terms of the original factors:

$$\begin{aligned} x_z &= mx_p x_q + i_p x_q + i_q x_p \\ &= (mx_p + i_p)x_q + i_q x_p \\ &= px_q + i_q x_p \\ &= (mx_q + i_q)x_p + i_p x_q \\ &= qx_p + i_p x_q. \end{aligned}$$

From the first form $x_z = px_q + i_q x_p$, we obtain:

$$x_z \equiv i_q x_p \pmod{p}.$$

Multiplying both sides by i_p (noting that $i_p \cdot i_q = i_z$ and $i_p^2 = 1$ since $i_p \in \{\pm 1\}$):

$$i_p x_z \equiv i_p i_q x_p \equiv i_z x_p \pmod{p}.$$

Since $\gcd(i_p, p) = 1$ (as $i_p \in \{\pm 1\}$), we can equivalently write:

$$x_z \equiv \frac{i_z}{i_p} x_p \pmod{p},$$

which establishes the desired equivalence relation.

Application to the Xp-Identity: This equivalence allows us to identify all composite indices in the X-arrays by solving for x_z modulo p. For a prime $p = iZ(x_p, i_p)$, its multiples in each residue class can be marked additively with step size p, starting from the appropriate initial index determined by this relation. This forms the mathematical foundation for the efficient composite marking in SiZ and SiZm algorithms. □

References

[1] Wikipedia contributors. Sieve of Eratosthenes. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes`

[2] Wikipedia contributors. Segmented sieve. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes#Segmented_sieve`

[3] Wikipedia contributors. Euler's sieve. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes#Euler's_sieve`

[4] Wikipedia contributors. Sieve of Sundaram. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Sundaram`

[5] Wikipedia contributors. Sieve of Atkin. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Atkin`